

Application Python

Contexte

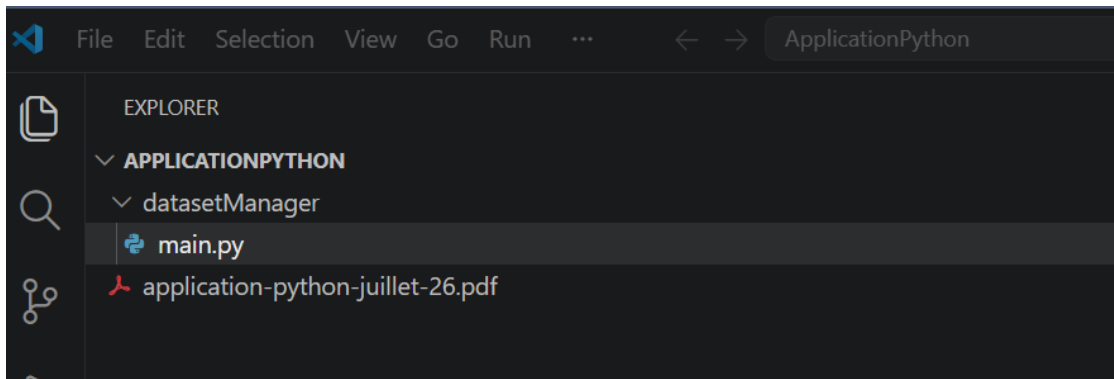
Vous venez d'intégrer une entreprise spécialisée dans l'Intelligence Artificielle.

Les Data Scientists manipulent quotidiennement des centaines de datasets (CSV et JSON). Avant de commencer les traitements avec Pandas, ils souhaitent disposer d'une application Python permettant de gérer ces jeux de données.

Votre mission consiste à développer progressivement une application console permettant de :

- gérer un catalogue de jeux de données ;
- enregistrer leurs caractéristiques ;
- effectuer des recherches ;
- afficher des statistiques ;
- sauvegarder les informations dans des fichiers ; ☐ charger automatiquement les données enregistrées ; ☐ gérer les erreurs d'utilisation.

- 1) Créez datasetManager, le dossier racine de l'application
- 2) Créez le fichier main.py, point d'entrée de l'application, pour contenir tout le code de la partie 1 à la partie 8.



Partie 1 : Types de base, variables, Entrées et sorties

- 3) Demandez à l'utilisateur de saisir les métadonnées d'un dataset :
 - nom du dataset
 - domaine
 - nombre de lignes
 - nombre de colonnes
 - taille en Mo
 - format (csv ou json)
 - public (true ou false)
- 4) Affichez ensuite un résumé formaté.

```
File Edit Selection View Go Run ... ApplicationPython
main.py x
datasetManager > main.py
1 # Partie 1 : Types de base, variables, Entrées et sorties
2 # Saisie des métadonnées
3 nom = input("Nom du dataset : ")
4 domaine = input("Domaine : ")
5
6 nb_lignes = int(input("Nombre de lignes : "))
7 nb_colonnes = int(input("Nombre de colonnes : "))
8 taille = float(input("Taille (en Mo) : "))
9
10 # Vérification du format
11 while True:
12     format_fichier = input("Format (csv ou json) : ").lower()
13     if format_fichier == "csv" or format_fichier == "json":
14         break
15     else:
16         print("Erreur : le format doit être csv ou json.")
17
18 # Vérification de public
19 while True:
20     public_input = input("Public (true ou false) : ").lower()
21     if public_input == "true":
22         public = True
23         break
24     elif public_input == "false":
25         public = False
26         break
27     else:
28         print("Erreur : veuillez saisir true ou false.")
29
```

```
29
30 # Affichage du résumé
31 print("\n RÉSUMÉ DU DATASET ")
32 print(f"Nom : {nom}")
33 print(f"Domaine : {domaine}")
34 print(f"Nombre de lignes : {nb_lignes}")
35 print(f"Nombre de colonnes : {nb_colonnes}")
36 print(f"Taille : {taille} Mo")
37 print(f"Format : {format_fichier}")
38 print(f"Public : {public}")

```

```
> ▼ TERMINAL
PS G:\ECSAIA\ApplicationPython> py main.py
C:\Users\other\AppData\Local\Python\pythoncore-3.14-64\python.exe: can't open file 'G:\\ECSAIA\\ApplicationPython\\main.py': [Errno 2] No such file or directory
PS G:\ECSAIA\ApplicationPython> cd .\datasetManager\
PS G:\ECSAIA\ApplicationPython\datasetManager> py main.py
Nom du dataset : Patient
Domaine : Medecine
Nombre de lignes : 1000
Nombre de colonnes : 10
Taille (en Mo) : 10
Format (csv ou json) : csv
Public (true ou false) : true

RÉSUMÉ DU DATASET
Nom : Patient
Domaine : Medecine
Nombre de lignes : 1000
Nombre de colonnes : 10
Taille : 10.0 Mo
Format : csv
Public : True
PS G:\ECSAIA\ApplicationPython\datasetManager> 
```

Partie 2 – Structures de contrôle

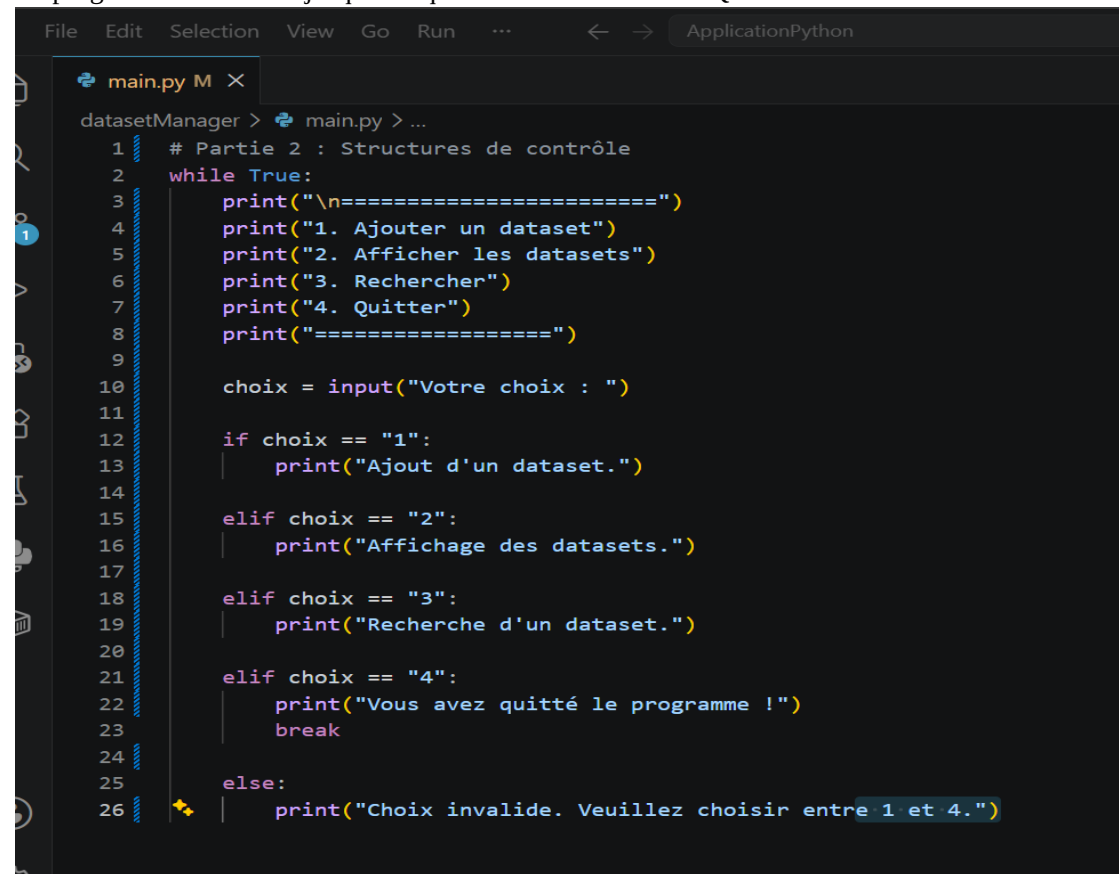
5) Créez un menu interactif (provisoire)

=====

1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter

=====

Le programme reste actif jusqu'à ce que l'utilisateur choisisse Quitter.



```
File Edit Selection View Go Run ... ApplicationPython
main.py M X
datasetManager > main.py > ...
1 # Partie 2 : Structures de contrôle
2 while True:
3     print("\n=====")
4     print("1. Ajouter un dataset")
5     print("2. Afficher les datasets")
6     print("3. Rechercher")
7     print("4. Quitter")
8     print("=====")
9
10    choix = input("Votre choix : ")
11
12    if choix == "1":
13        print("Ajout d'un dataset.")
14
15    elif choix == "2":
16        print("Affichage des datasets.")
17
18    elif choix == "3":
19        print("Recherche d'un dataset.")
20
21    elif choix == "4":
22        print("Vous avez quitté le programme !")
23        break
24
25    else:
26        print("Choix invalide. Veuillez choisir entre 1 et 4.")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

▼ TERMINAL

● PS G:\ECSAIA\ApplicationPython\datasetManager> py main.py

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 1
Ajout d'un dataset.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 2
Affichage des datasets.

=====
1. Ajouter un dataset
2. Afficher les datasets
3. Rechercher
4. Quitter
=====
Votre choix : 4
Vous avez quitté le programme !
○ PS G:\ECSAIA\ApplicationPython\datasetManager> █
```

Partie 3 – Dictionnaires

6) Créez un dictionnaire pour stocker les métadonnées de chaque dataset

Exemple :

```
{  
  
"nom": "Titanic",  
"domaine": "Transport",  
"lignes": 891,  
"colonnes": 12,  
"taille": 48,  
"format": "CSV",  
"public": True  
}
```

```
dataset = {  
    "nom": nom,  
    "domaine": domaine,  
    "lignes": lignes,  
    "colonnes": colonnes,  
    "taille": taille,  
    "format": format,  
    "public": public  
}
```

Partie 4 – Tuples

7) Créez un tuple contenant les domaines autorisés.

("Santé", "Finance", "Agriculture", "Transport", "Education")

```
domaine_Autorise = ("Santé", "Finance", "Agriculture", "Transport", "Education")
```

8) Vérifiez que le domaine saisi, à la question 3, appartient au tuple.

```
if domaine not in domaines_autorises:  
    print(f"Domaine invalide. Domaines autorisés : {domaines_autorises}\n")
```

```
Votre choix : 1  
Nom du dataset : Medecine  
Domaine : Q  
Domaine invalide. Domaines autorisés : ('Santé', 'Finance', 'Agriculture', 'Transport', 'Education')
```

Partie 5 – Listes

9) Créez une liste contenant les datasets. Chaque ajout est enregistré dans la liste

```
datasets = []
```

```
datasets.append(dataset)
```

10) Ajoutez les fonctionnalités :

- ajouter
- trier
- rechercher
- modifier
- supprimer

```
elif choix == "2":
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
    else:
        print(f"\n=== Liste des datasets ({len(datasets)}) ===")
        for i, d in enumerate(datasets, start=1):
            print(f"\n--- Dataset {i} ---")
            print(f"Nom       : {d['nom']}")
            print(f"Domaine  : {d['domaine']}")
            print(f"Lignes   : {d['lignes']}")
            print(f"Colonnes : {d['colonnes']}")
            print(f"Taille   : {d['taille']} Mo")
            print(f"Format   : {d['format']}")
            print(f"Public   : {'Oui' if d['public'] else 'Non'}")
        print("=====\n")
```

```
PS G:\ECSAIA\ApplicationPython\datasetManager> py main.py
Votre choix : 2
```

```
=== Liste des datasets (1) ===
```

```
--- Dataset 1 ---
```

```
Nom       : Medecin
```

```
Domaine   : Santé
```

```
Lignes    : 40
```

```
Colonnes  : 3
```

```
Taille    : 3.0 Mo
```

```
Format    : JSON
```

```
Public    : Oui
```

```
=====
```

```

elif choix == "3":
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
    else:
        nom_recherche = input("Nom du dataset à rechercher : ")
        trouve = False
        for d in datasets:
            if d["nom"].lower() == nom_recherche.lower():
                print(f"\n=== Dataset trouvé ===")
                print(f"Nom      : {d['nom']}")
                print(f"Domaine  : {d['domaine']}")
                print(f"Lignes   : {d['lignes']}")
                print(f"Colonnes  : {d['colonnes']}")
                print(f"Taille   : {d['taille']} Mo")
                print(f"Format    : {d['format']}")
                print(f"Public    : {'Oui' if d['public'] else 'Non'}")
                print("=====\n")
                trouve = True
                break
        if not trouve:
            print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")

```

```

Votre choix : 3
Nom du dataset à rechercher : Medecin

=== Dataset trouvé ===
Nom      : Medecin
Domaine  : Santé
Lignes   : 40
Colonnes  : 3
Taille   : 3.0 Mo
Format    : JSON
Public    : Oui
=====

```

```

elif choix == "4":
    print("Vous avez quitté le programme !")
    break
elif choix == "5":
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
    else:
        datasets_tries = sorted(datasets, key=lambda d: d["lignes"])
        print(
            f"\n=== Datasets triés par nombre de lignes ({len(datasets_tries)}) ==="
        )
        for i, d in enumerate(datasets_tries, start=1):
            print(f"{i}. {d['nom']} - {d['lignes']} lignes")
        print("=====\n")

```

```

=====
Votre choix : 5

=== Datasets triés par nombre de lignes (1) ===
1. Medecin - 40 lignes
=====

```

```

    print("=====\n")
elif choix == "6":
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
    else:
        nom_recherche = input("Nom du dataset à modifier : ")
        trouve = False
        for d in datasets:
            if d["nom"].lower() == nom_recherche.lower():
                trouve = True
                print(f"\nDataset '{d['nom']}' trouvé. Laissez vide pour ne pas changer.")

                nouveau_domaine = input(f"Domaine ({d['domaine']}) : ")
                if nouveau_domaine != "":
                    if nouveau_domaine not in domaines_autorises:
                        print(f"Domaine invalide, conservé : {d['domaine']}")
                    else:
                        d["domaine"] = nouveau_domaine

                nouvelles_lignes = input(f"Lignes ({d['lignes']}) : ")
                if nouvelles_lignes != "":
                    d["lignes"] = int(nouvelles_lignes)

                nouvelles_colonnes = input(f"Colonnes ({d['colonnes']}) : ")
                if nouvelles_colonnes != "":
                    d["colonnes"] = int(nouvelles_colonnes)

                nouvelle_taille = input(f"Taille ({d['taille']} Mo) : ")
                if nouvelle_taille != "":
                    d["taille"] = float(nouvelle_taille)

                print(f"\nDataset '{d['nom']}' mis à jour avec succès.\n")
                break

        if not trouve:
            print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")
elif choix == "7":

```



```
=====
Votre choix : 6
Nom du dataset à modifier : Medecin

Dataset 'Medecin' trouvé. Laissez vide pour ne pas changer.
Domaine (Santé) : Finance
Lignes (40) :
Colonnes (3) :
Taille (3.0 Mo) :

Dataset 'Medecin' mis à jour avec succès.
```

```
print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")
elif choix == "7":
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
    else:
        nom_recherche = input("Nom du dataset à supprimer : ")
        trouve = False
        for d in datasets:
            if d["nom"].lower() == nom_recherche.lower():
                datasets.remove(d)
                print(f"\nDataset '{d['nom']}' supprimé avec succès.\n")
                trouve = True
                break
        if not trouve:
            print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")
    else:
        print("Choix invalide. Veuillez choisir entre 1 et 7.")
```

```
=====
Votre choix : 7
Nom du dataset à supprimer : Medecin

Dataset 'Medecin' supprimé avec succès.
```

Partie 6 – Compréhensions (Listes et Dictionnaires)

11) Affichez les statistiques comme par exemple :

- Nombre de datasets : 18
- Nombre total de lignes : 2 540 000
- Nombre moyen de colonnes : 27
- Datasets publics : 8
- Datasets privés : 10
- Nombre de datasets au format CSV : 13 Nombre de datasets au format JSON : 5
- Répartition par domaine o Santé : 5 o Finance : 3 o Agriculture : 2 o
Transport : 4 o Education : 4

Testez, au fur et à mesure, avec des datasets d'exemples

```
elif choix == "8":
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
    else:
        nb_datasets = len(datasets)
        total_lignes = sum(d["lignes"] for d in datasets)
        moyenne_colonnes = sum(d["colonnes"] for d in datasets) / nb_datasets
        nb_publics = len([d for d in datasets if d["public"]])
        nb_prives = len([d for d in datasets if not d["public"]])
        nb_csv = len([d for d in datasets if d["format"] == "CSV"])
        nb_json = len([d for d in datasets if d["format"] == "JSON"])

        # Répartition par domaine (compréhension de dictionnaire)
        repartition_domaines = {
            domaine: len([d for d in datasets if d["domaine"] == domaine])
            for domaine in domaines_autorises
            if any(d["domaine"] == domaine for d in datasets)
        }

        print("\n=== Statistiques ===")
        print(f"Nombre de datasets      : {nb_datasets}")
        print(f"Nombre total de lignes    : {total_lignes}")
        print(f"Nombre moyen de colonnes  : {moyenne_colonnes:.0f}")
        print(f"Datasets publics         : {nb_publics}")
        print(f"Datasets privés          : {nb_prives}")
        print(f"Format CSV               : {nb_csv}")
        print(f"Format JSON              : {nb_json}")
        print("Répartition par domaine :")
        for domaine, nb in repartition_domaines.items():
            print(f"    {domaine} : {nb}")
        print("=====\n")
```

```

7. Supprimer
8. Statistiques
=====
Votre choix : 8

=== Statistiques ===
Nombre de datasets      : 4
Nombre total de lignes  : 2515891
Nombre moyen de colonnes : 14
Datasets publics       : 3
Datasets privés        : 1
Format CSV              : 3
Format JSON             : 1
Répartition par domaine :
    Santé : 1
    Finance : 1
    Agriculture : 1
    Transport : 1
=====

```

Partie 7 – Fichiers

12) Créez le fichier datasets.csv pour :

- sauvegarder les données

```

print("=====\n")
elif choix == "9":
    if len(datasets) == 0:
        print("\nAucun dataset à sauvegarder.\n")
    else:
        fichier = open("datasets.csv", "w", encoding="utf-8")
        fichier.write("nom, domaine, lignes, colonnes, taille, format, public\n")
        for d in datasets:
            ligne = f"{d['nom']},{d['domaine']},{d['lignes']},{d['colonnes']},{d['taille']},{d['format']},{d['public']}\n"
            fichier.write(ligne)
        fichier.close()
        print(f"\n{len(datasets)} dataset(s) sauvegardé(s) dans datasets.csv\n")

```

```
main.py M datasets.csv U X
datasetManager > datasets.csv
1 nom,domaine,lignes,colonnes,taille,format,public
2 Titanic,Transport,891,12,48.0,CSV,True
3 COVID-Cases,Santé,500000,8,120.5,JSON,True
4 Stock-Prices,Finance,2000000,15,300.0,CSV,False
5 Crop-Yields,Agriculture,15000,20,25.0,CSV,True
6
```

- recharger et afficher les données

```
print(f"\n{len(datasets)} dataset(s) sauvegardé(s) dans datasets.csv\n")
elif choix == "10":
    fichier = open("datasets.csv", "r", encoding="utf-8")
    lignes_fichier = fichier.readlines()
    fichier.close()

    datasets = []
    for ligne in lignes_fichier[1:]: # on saute l'en-tête
        valeurs = ligne.strip().split(",")
        dataset = {
            "nom": valeurs[0],
            "domaine": valeurs[1],
            "lignes": int(valeurs[2]),
            "colonnes": int(valeurs[3]),
            "taille": float(valeurs[4]),
            "format": valeurs[5],
            "public": valeurs[6] == "True",
        }
        datasets.append(dataset)

    print(f"\n{len(datasets)} dataset(s) rechargé(s) depuis datasets.csv\n")
```

```
=====
Votre choix : 10

4 dataset(s) rechargé(s) depuis datasets.csv
```

Partie 8 – Exceptions Plusieurs

erreurs sont constatées :

- l'utilisateur saisit un texte au lieu d'un nombre ;
- le fichier n'existe pas ;
- le dataset recherché n'existe pas ; ☐ le fichier est vide.

13) Gérez ces exceptions pour que le programme continue de fonctionner.

```
try:
    lignes = int(input("Nombre de lignes : "))
    colonnes = int(input("Nombre de colonnes : "))
    taille = float(input("Taille en Mo : "))
except ValueError:
    print(
        "\nErreur : veuillez saisir un nombre valide pour lignes/colonnes/taille.\n"
    )

fichier = open("datasets.csv", "r", encoding="utf-8")
lignes_fichier = fichier.readlines()
fichier.close()

if len(lignes_fichier) <= 1:
    print("\nErreur : le fichier datasets.csv est vide.\n")
else:
    datasets = []
    for ligne in lignes_fichier[1:]:
        valeurs = ligne.strip().split(",")
        dataset = {
            "nom": valeurs[0],
            "domaine": valeurs[1],
            "lignes": int(valeurs[2]),
            "colonnes": int(valeurs[3]),
            "taille": float(valeurs[4]),
            "format": valeurs[5],
            "public": valeurs[6] == "True",
        }
        datasets.append(dataset)

    print(f"\n{len(datasets)} dataset(s) rechargé(s) depuis datasets.csv\n")

except FileNotFoundError:
    print(
        "\nErreur : le fichier datasets.csv n'existe pas. Sauvegardez d'abord (option 9).\n"
    )
```

```
TERMINAL
PS G:\ECSAIA\ApplicationPython\datasetManager> py main.py
Votre choix : 1
Nom du dataset : Medecin
Domaine : Santé
Nombre de lignes : ABC

Erreur : veuillez saisir un nombre valide pour lignes/colonnes/taille.

=====
Votre choix : 10

Erreur : le fichier datasets.csv n'existe pas. Sauvegardez d'abord (option 9).

=====
Votre choix : 3
Nom du dataset à rechercher : bbù

Aucun dataset trouvé avec le nom 'bbù'.
```

Partie 9 – Fonctions

14) Refactorisez le programme en créant les fonctions suivantes :

afficher_menu() ajouter_dataset()
afficher_datasets()
supprimer_dataset()
rechercher_dataset() trier_dataset()
modifier_dataset() sauvegarder()
recharger()
statistiques()

```
def afficher_menu():  
    print("\n=====")  
    print("1. Ajouter un dataset")  
    print("2. Afficher les datasets")  
    print("3. Rechercher")  
    print("4. Quitter")  
    print("5. Trier")  
    print("6. Modifier")  
    print("7. Supprimer")  
    print("8. Statistiques")  
    print("9. Sauvegarder")  
    print("10. Recharger")  
    print("=====")
```

```
def afficher_datasets():  
    if len(datasets) == 0:  
        print("\nAucun dataset enregistré pour le moment.\n")  
        return  
  
    print(f"\n== Liste des datasets ({len(datasets)}) ==")  
    for i, d in enumerate(datasets, start=1):  
        print(f"\n--- Dataset {i} ---")  
        print(f"Nom       : {d['nom']}")  
        print(f"Domaine    : {d['domaine']}")  
        print(f"Lignes     : {d['lignes']}")  
        print(f"Colonnes   : {d['colonnes']}")  
        print(f"Taille     : {d['taille']} Mo")  
        print(f"Format     : {d['format']}")  
        print(f"Public     : {'Oui' if d['public'] else 'Non'}")  
    print("=====\n")
```

```

def rechercher_dataset():
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
        return

    nom_recherche = input("Nom du dataset à rechercher : ")
    for d in datasets:
        if d["nom"].lower() == nom_recherche.lower():
            print(f"\n=== Dataset trouvé ===")
            print(f"Nom      : {d['nom']}")
            print(f"Domaine   : {d['domaine']}")
            print(f"Lignes    : {d['lignes']}")
            print(f"Colonnes   : {d['colonnes']}")
            print(f"Taille     : {d['taille']} Mo")
            print(f"Format     : {d['format']}")
            print(f"Public     : {'Oui' if d['public'] else 'Non'}")
            print("=====\n")
            return

    print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")

def trier_dataset():
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
        return

    datasets_tries = sorted(datasets, key=lambda d: d["lignes"])
    print(f"\n=== Datasets triés par nombre de lignes ({len(datasets_tries)}) ===")
    for i, d in enumerate(datasets_tries, start=1):
        print(f"{i}. {d['nom']} - {d['lignes']} lignes")
    print("=====\n")

def modifier_dataset():
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
        return

    nom_recherche = input("Nom du dataset à modifier : ")
    for d in datasets:
        if d["nom"].lower() == nom_recherche.lower():
            print(f"\nDataset '{d['nom']}' trouvé. Laissez vide pour ne pas changer.")
            nouveau_domaine = input(f"Domaine ({d['domaine']}) : ")
            if nouveau_domaine != "":
                if nouveau_domaine not in domaines_autorises:
                    print(f"Domaine invalide, conservé : {d['domaine']}")
                else:
                    d["domaine"] = nouveau_domaine

            nouvelles_lignes = input(f"Lignes ({d['lignes']}) : ")
            if nouvelles_lignes != "":
                d["lignes"] = int(nouvelles_lignes)

            nouvelles_colonnes = input(f"Colonnes ({d['colonnes']}) : ")
            if nouvelles_colonnes != "":
                d["colonnes"] = int(nouvelles_colonnes)

            nouvelle_taille = input(f"Taille ({d['taille']} Mo) : ")
            if nouvelle_taille != "":
                d["taille"] = float(nouvelle_taille)

            print(f"\nDataset '{d['nom']}' mis à jour avec succès.\n")
            return

    print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")

```

```

def supprimer_dataset():
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
        return

    nom_recherche = input("Nom du dataset à supprimer : ")
    for d in datasets:
        if d["nom"].lower() == nom_recherche.lower():
            datasets.remove(d)
            print(f"\nDataset '{d['nom']}' supprimé avec succès.\n")
            return

    print(f"\nAucun dataset trouvé avec le nom '{nom_recherche}'.\n")

def statistiques():
    if len(datasets) == 0:
        print("\nAucun dataset enregistré pour le moment.\n")
        return

    nb_datasets = len(datasets)
    total_lignes = sum(d["lignes"] for d in datasets)
    moyenne_colonnes = sum(d["colonnes"] for d in datasets) / nb_datasets
    nb_publics = len([d for d in datasets if d["public"]])
    nb_prives = len([d for d in datasets if not d["public"]])
    nb_csv = len([d for d in datasets if d["format"] == "CSV"])
    nb_json = len([d for d in datasets if d["format"] == "JSON"])

    repartition_domaines = {
        domaine: len([d for d in datasets if d["domaine"] == domaine])
        for domaine in domaines_autorises
        if any(d["domaine"] == domaine for d in datasets)
    }

    print("\n=== Statistiques ===")
    print(f"Nombre de datasets      : {nb_datasets}")
    print(f"Nombre total de lignes   : {total_lignes}")
    print(f"Nombre moyen de colonnes : {moyenne_colonnes:.0f}")
    print(f"Datasets publics        : {nb_publics}")
    print(f"Datasets privés         : {nb_prives}")
    print(f"Format CSV              : {nb_csv}")
    print(f"Format JSON             : {nb_json}")
    print("Répartition par domaine :")
    for domaine, nb in repartition_domaines.items():
        print(f"  {domaine} : {nb}")
    print("=====\n")

```



```

manager 2 • main.py • modmer_dataset
def sauvegarder():
    if len(datasets) == 0:
        print("\nAucun dataset à sauvegarder.\n")
        return

    fichier = open("datasets.csv", "w", encoding="utf-8")
    fichier.write("nom, domaine, lignes, colonnes, taille, format, public\n")
    for d in datasets:
        ligne = f"{d['nom']},{d['domaine']},{d['lignes']},{d['colonnes']},{d['taille']},{d['format']},{d['public']}\n"
        fichier.write(ligne)
    fichier.close()
    print(f"\n{len(datasets)} dataset(s) sauvegardé(s) dans datasets.csv\n")

def recharger():
    global datasets
    try:
        fichier = open("datasets.csv", "r", encoding="utf-8")
        lignes_fichier = fichier.readlines()
        fichier.close()

        if len(lignes_fichier) <= 1:
            print("\nErreur : le fichier datasets.csv est vide.\n")
            return

        nouveaux_datasets = []
        for ligne in lignes_fichier[1:]:
            valeurs = ligne.strip().split(",")
            dataset = {
                "nom": valeurs[0],
                "domaine": valeurs[1],
                "lignes": int(valeurs[2]),
                "colonnes": int(valeurs[3]),
                "taille": float(valeurs[4]),
                "format": valeurs[5],
                "public": valeurs[6] == "True",
            }
            nouveaux_datasets.append(dataset)

        datasets = nouveaux_datasets
        print(f"\n{len(datasets)} dataset(s) rechargé(s) depuis datasets.csv\n")

    except FileNotFoundError:
        print(
            "\nErreur : le fichier datasets.csv n'existe pas. Sauvegardez d'abord (opti
        )

```

```

def main():
    while True:
        afficher_menu()
        choix = input("Votre choix : ")

        if choix == "1":
            ajouter_dataset()
        elif choix == "2":
            afficher_datasets()
        elif choix == "3":
            rechercher_dataset()
        elif choix == "4":
            print("Vous avez quitté le programme !")
            break
        elif choix == "5":
            trier_dataset()
        elif choix == "6":
            modifier_dataset()
        elif choix == "7":
            supprimer_dataset()
        elif choix == "8":
            statistiques()
        elif choix == "9":
            sauvegarder()
        elif choix == "10":
            recharger()
        else:
            print("Choix invalide. Veuillez choisir entre 1 et 10.")

if __name__ == "__main__":
    main()

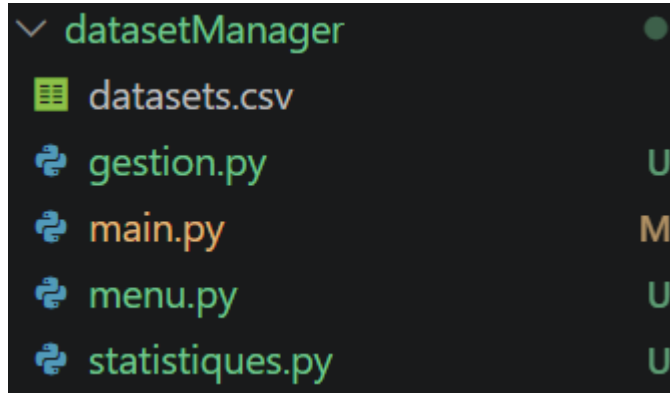
```

Partie 10 – Modules

Jusqu'à présent, toute l'application est contenue dans un seul fichier.

15) Pour éviter des problèmes de maintenance, répartissez le code dans plusieurs modules.

```
main.py menu.py
gestion.py
statistiques.py
```



Partie 11 – Packages

16) Découpez l'application en packages comme suit :

```
datasetManager/ |—
```

```
main.py
```

```
|— datasets/
```

```
|   __init__.py
```

```
|   gestion.py
```

```
|   statistiques.py
```

```
|— interface/
```

```
|   __init__.py
```

```
|   menu.py
```

```
|   affichage.py
```

```
|— stockage/
```

```
|   __init__.py
```

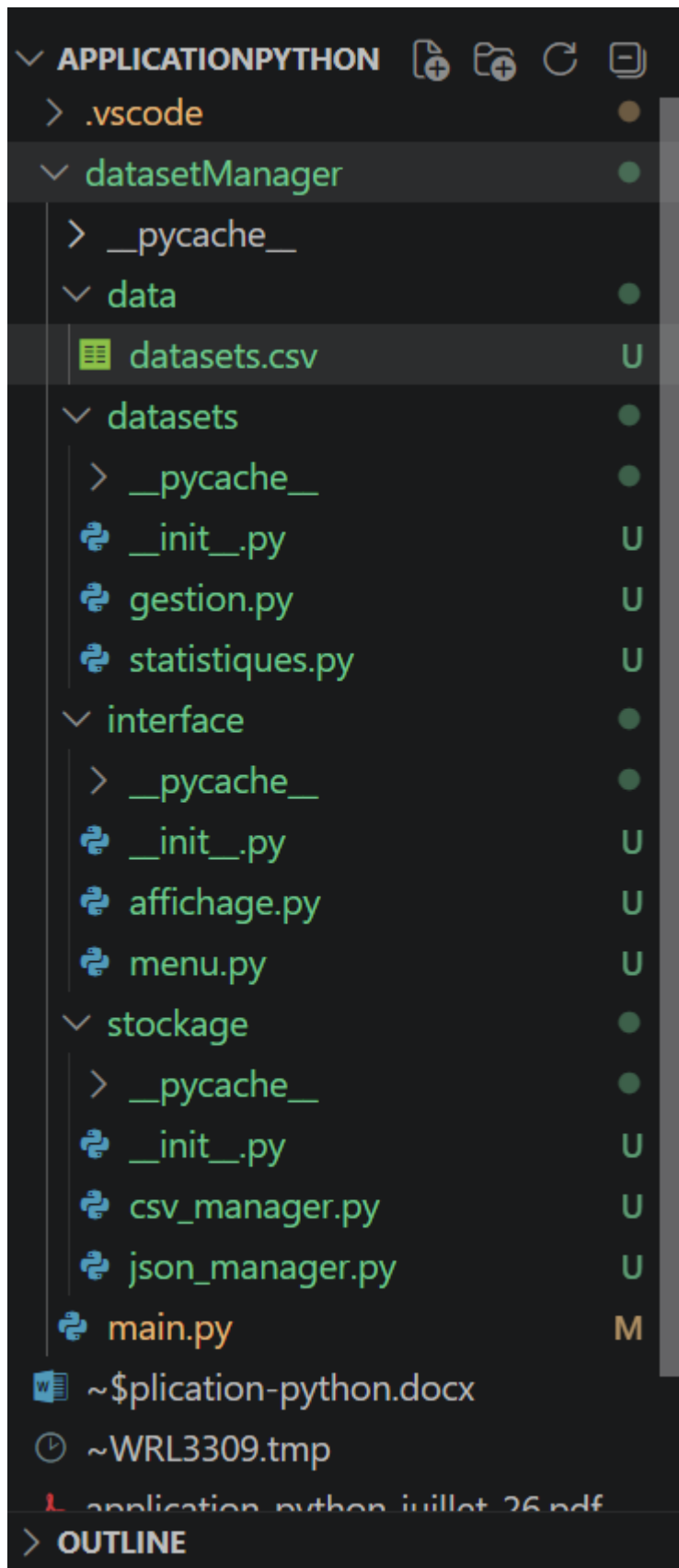
```
|   csv_manager.py
```

```
|   json_manager.py
```

```
|— data/
```

```
|   datasets.csv
```

```
|   datasets.json
```



Partie 12 : Bonus

Proposez et réalisez toute fonctionnalité pertinente et utile qu'on pourrait rajouter à l'application.

En plus de la sauvegarde/rechargement CSV demandés en Partie 7, une fonctionnalité d'export et d'import au format JSON a été ajoutée (module stockage/json_manager.py), ce qui justifie la présence de data/datasets.json dans l'arborescence imposée par le sujet. Le menu propose ainsi deux formats de sauvegarde au choix.

```
print("11. Sauvegarder (JSON)")
print("12. Recharger (JSON)")

    datasets.extend(resultat)
elif choix == "11":
    sauvegarder_json(datasets)
elif choix == "12":
    resultat = charger_json()
    if resultat is not None:
        datasets.clear()
        datasets.extend(resultat)
```

```
datasetManager > stockage > json_manager.py > ...
import json

def sauvegarder_json(datasets, chemin="data/datasets.json"):
    if len(datasets) == 0:
        print("\nAucun dataset à sauvegarder.\n")
        return

    fichier = open(chemin, "w", encoding="utf-8")
    json.dump(datasets, fichier, indent=4, ensure_ascii=False)
    fichier.close()
    print(f"\n{len(datasets)} dataset(s) sauvegardé(s) dans {chemin}\n")

def charger_json(chemin="data/datasets.json"):
    try:
        fichier = open(chemin, "r", encoding="utf-8")
        nouveaux_datasets = json.load(fichier)
        fichier.close()

        if len(nouveaux_datasets) == 0:
            print("\nErreur : le fichier JSON est vide.\n")
            return None

        print(f"\n{len(nouveaux_datasets)} dataset(s) rechargé(s) depuis {chemin}\n")
        return nouveaux_datasets

    except FileNotFoundError:
        print(f"\nErreur : le fichier {chemin} n'existe pas. Sauvegardez d'abord.\n")
        return None
```

```
=====
Votre choix : 11

2 dataset(s) sauvegardé(s) dans data/datasets.json
```

Conclusion

Ce projet a permis de construire progressivement une application console de gestion de datasets, des bases de Python (variables, structures de contrôle, dictionnaires, tuples, listes, fichiers, exceptions) jusqu'à une architecture organisée en fonctions, modules puis packages. La fonctionnalité bonus d'export/import JSON complète la sauvegarde CSV déjà en place.